NEUTRAL TASK BROKER · ARCHITECTURE

OpenVow 架构图 v3.0

人定契约，终端干活，平台只做派发、校验、归集。平台不持有任何模型凭证，也从不主动发起模型调用； 连拆解、写测试、合并都是终端拉的活 —— 判卷的人，从来不是考生。



控制面 · 人（定契约） · 调度面 · 平台（无模型） · 产物库（不可变） · 执行面 · 平台外 ---- 信任边界

任务状态机

只有 **accepted** 解锁下游；**merged** 需要集成任务通过。状态以租约为唯一时钟。

queued
待认领 · 依赖已满足

→

claimed
已加租约 · 心跳续租

→

validating
L0–L2 结构与形状

→

submitted
等 L3 验收证据

→

accepted
可用 · 解锁下游

→

merged
已进主干

异常出口： `rejected` L0–L3 失败，计入 attempts，可重试； `lease_expired` 终端失联 → 回到 queued； `attempts > max` → `needs_human`，不再自动派发； `stale` base_commit 落后且写集相交 → 要求 rebase 重提交。

任务流水线：每条产出都是任务

平台里没有隐含动作。拆解、冻结接口、写测试、写实现、合并、修复，全部走同一套 claim / 校验 / submit； **同一个终端可以领任何一种**，区别只在契约。

decompose
需求 → 图 · 人审

→

interface
骨架 · 独占只读

→

test
必须红着交

→

implement
必须绿着交

→

integration
全量构建 + 全量测

→

merged
进 canonical 主干

集成失败 ⇒ 自动生成 `repair` 任务（优先给没参与产出的终端）→ 修完重跑，3 次不过升级 `needs_human`。 合并是一次真实执行，不是一个勾选步骤。

kind	写集	验收语义	依赖
`interface`	接口文件（全图只读）	**骨架必须能编译** · skeleton_build	—
`test`	测试 + fixtures	**必须在骨架上失败** · red_tests	interface
`implement`	实现文件（不含接口/测试）	**冻结测试必须全绿** · green_tests	test
`integration`	全仓（受控）	**全量构建 + 全量测** · full	组内全 accepted
`repair`	冲突相关文件	**全量通过** · full	integration failed

01
判卷的人不是考生
测试由独立任务产出，复跑时用冻结版本覆盖提交版本

02
空测试过不了红测试
`assert!(true)` 在骨架上会通过 → 直接拒单

03
接口改不动
interface 被 accepted 后全图只读，写集命中即拒

04
越界交不进来
提交路径必须 ⊆ writes，且不含冻结文件

四层校验 + 一次真实合并

校验只看产物；合并由 integration 任务承担 —— 它要真的跑起来，不是打个勾。

L0
结构
路径 ⊆ writes · 无越界 / 穿越 / 符号链接 · 大小上限

L1
完整
重算 sha256 · required 齐全 · 冻结文件未被改动

L2
形状
JSON Schema · linter 零 error

L3
行为 · acceptance
按 kind 跑 build / test；平台按信任分复跑

I
集成 · 任务
容器内全量构建 + 全量测，失败生成 repair

L0–L3 失败 ⇒ **rejected**；通过 ⇒ **accepted**（解锁下游）； 集成通过 ⇒ **merged**。只信退出码与测试结果，不信终端自报的 `status`。

拆解层 · 拆解本身也是一个任务

平台内置拆解模型 = 把风控重新集中回平台。正确形态是让一个自带强模型凭证的 **planner 终端** 去 claim 一个 meta 任务， 产出机器可校验的 `task_graph.json`：图里必须出现 interface 与 test 节点，否则 V8 直接拒；人只对图签字。

需求 + meta 任务
label: decompose

→

Planner 终端
claim · 自带强模型

→

graph 草稿
结构化 DAG

→

validator
V1–V8 · 纯函数

→

人审 approve
只看变更节点

→

active 图
子任务入 queued

回环： `reject` 带 reason 回到 planner 重拆（新 version，绝不覆盖旧图）； `acceptance_trivial` 拒绝 `exit 0` / `|| true` 这类空命令； 图只能被新 version `supersede` ，已 accepted 的产物只标 `stale` ，永不删除。

V1
结构
依赖无环 · id 唯一 · 引用存在

V2
所有权
writes 两两不相交，路径规范化

V3
完备
每节点有验收命令，且不是空命令

V4
覆盖
每条需求至少被一个节点 covers

V5
粒度
≤ 4 文件 / ≤ 400 行 / ≤ 12 节点

V6
基线
base_commit 存在，不与在途任务冲突

V7
接口闭合
每个 consumes 都能在 provides 或基线里找到

V8
类型拓扑
implement 的祖先必须有 interface 与 test

```
{
  "nodes": [{
    "id": "N-01",
    "writes": ["src/parser/**"],
    "covers": ["R-2026-014/1.1"],
    "acceptance": { "kind": "green_tests",
                    "build": "cargo build --locked",
                    "run": "cargo test --locked parser" },
    "est": { "files": 3, "diff_lines": 260 }
  }],
  "decisions": [{ "id": "D-1", "text": "手写递归下降，不引入生成器" }],
  "uncertain": ["需求 3.2 未说明错误恢复策略"]
}
```

01 判卷的不是考生

测试由独立任务产出并被冻结，复跑时覆盖提交版本。实现方唯一能改的只有实现文件——它没法给自己打分。

02 写集不相交

两个任务都改同一个文件，各自 100% 合规，合并必然冲突。writes 相交 ⇒ 禁止并行，宁可少并行，不要假并行。

03 冻结即真相

接口、测试、验收命令一旦 accepted 就不可改。想改只能走图的新版本，下游标 stale 重来——不允许静默覆盖。

04 模型 Key 永不上平台

平台只持终端接入 token。两种凭证必须分开叙述——否则「不持有任何密钥」和「防投毒」会直接打架。